GEORGI KRASTEV

Senior Full-Stack Engineer | Technical Architect | Platform Modernization Specialist

Sofia, Bulgaria | +359 87 960 6986 | krustevgeorgi@yahoo.com | Interactive Portfolio

---

PROFESSIONAL SUMMARY

Hands-on technical architect with 9+ years architecting and shipping scalable, high-impact platforms across **real-time systems** (satellites), **industrial automation** (robots), **fintech integrations** (payments), and **AI-driven automation**. Proven expertise in **legacy platform modernization**, **design system leadership**, **third-party integrations at scale**, and **rapid MVP delivery** (3–4x faster than industry standard).

Core competencies:

- Full-stack development (React/Vue/Node/AWS)
- System architecture & technical decision-making
- Design systems & component standardization
- Payment processing & SOAP/REST integrations
- Real-time data visualization & WebSocket systems
- Cloud infrastructure (AWS, GitHub Actions CI/CD)
- AI/automation workflows (n8n, custom nodes, LLMs)
- Cross-functional team leadership & vendor management

Known for: Shipping MVPs faster than competitors, solving complex integration problems, centralizing fragmented codebases, and driving measurable operational improvements (cost reduction, time savings, quality gains).

CORE TECHNICAL SKILLS

Frontend Architecture

React (Hooks, Context, advanced patterns), **Redux** (state management at scale), **TypeScript** (strict typing, interfaces), **Vue 3** (Composition API, modern patterns), **Vite** (build optimization), **Pinia** (Vue state), **Tailwind CSS** (utility-first styling), **Material UI**, **Storybook** (component documentation), **WebSockets** (real-time bidirectional communication)

Backend & Database

Node.js (server-side JavaScript, high concurrency), **Express.js** (HTTP framework), **MongoDB** (document database, flexible schema), **SQL** (relational databases, complex queries), **Supabase** (PostgreSQL + realtime APIs), **REST APIs** (design & implementation), **SOAP/XML** (legacy integrations)

Cloud & DevOps

Amazon Web Services (AWS): S3 (object storage), CloudFront (CDN), EC2 (compute), Cognito (identity), Lambda (serverless), RDS (managed database); **Docker** (containerization, multi-stage builds), **GitHub Actions** (CI/CD pipelines, multi-environment deployments), **AWS Authentication & User Management** (IAM, session handling)

AI & Automation

n8n (workflow orchestration, 50+ step workflows), **Custom JavaScript Nodes** (extending n8n with bespoke logic), **OpenAI API** (GPT integrations, prompt engineering), **Automated Reporting Agents** (data aggregation, trend analysis, PDF generation), **IVR & Voice AI** (AI-generated voice responses)

Architecture & Leadership

Design Systems (Storybook, component libraries, brand consistency), **Legacy Modernization** (rewriting monoliths, architectural refactors), **Third-Party Integrations** (payment processors, APIs, webhooks), **RBAC** (role-based access control), **OAuth 2.0** (secure authentication flows), **Reconciliation Logic** (financial data alignment), **Agile/Scrum**, **System Design**

PROFESSIONAL EXPERIENCE

PRINCIPAL FULL-STACK ENGINEER / SOLUTION ARCHITECT

Boutique Tech Consultancy (MVP Forge / ZenGroup Umbrella) | Sofia, Bulgaria | *January 2024 – Present*

Leading technical delivery of **AI automation services** and **rapid SaaS development** for clients, with focus on **workflow automation**, **payment integrations**, and **MVP velocity**.

Prezaredi.bg – Fuel Payments & Fleet Cards Platform ***(Feb 2024 – Present)***

Problem: Corporate clients and end-users needed a digital alternative to plastic fuel cards at petrol stations—reducing friction, enabling delayed payments, and providing real-time fleet

management.

Architecture & Scope:

- **Mobile App:** React Native (iOS/Android) for fleet card and go-card users; cross-platform native performance.
- **Backend:** Node.js/Express running on AWS EC2; **PostgreSQL** database for transactional consistency; RDS for managed persistence.
- **Payment Processor:** BORICA (Bulgaria's largest card payment provider) SOAP XML API for card issuance, printing, PIN delivery, and settlement.
- **Third-Party Integrations:** Petrol station provider APIs for fuel price feeds, transaction capture, and real-time balance updates.
- **CI/CD Infrastructure:** GitHub Actions end-to-end (dev, staging, production); automated testing, linting, and deployment gates.
- **Call Center Workflow:** Lost/stolen card blocking via IVR with **AI-generated voice responses** (natural language, no human agent required after hours).

Key Technical Decisions & Risk Mitigation:

- **Reconciliation Logic:** Solved critical matching problems where transaction amounts differed between Prezaredi and petrol station ledgers (e.g., 55€ vs 55.50€ during invoicing). Implemented **idempotent reconciliation engine** with tolerance thresholds and manual review queues.
- **SOAP/XML Handling:** Built XML parsing and error recovery for BORICA API (strict schema validation, retry logic, webhook verification).
- **Offline Sync:** React Native app caches card balance and transaction history; syncs on reconnect with conflict resolution.
- **Security:** AWS Cognito for user authentication; encrypted card data in transit (TLS 1.3); PCI scope reduction through AWS tokenization.

Metrics & Outcomes:

- **20 corporate clients** onboarded; **~400 active fleet cards** in circulation.
- **Thousands of transactions per month** processed; **99%+ uptime** maintained.
- **Reduced petrol station card issuance time** from 5–7 days to real-time digital activation.
- **Delayed payment + discount model** improved cash flow for corporate clients; increased adoption vs. cash-only competitors.

Team Coordination:

- Managed **~15 external developers, designers, and project managers** from vendor organization.
- Owned **architecture decisions**, **integration contracts**, **acceptance criteria**, and **quality gates**.

- Negotiated with **petrol station technical teams** and **C-level brand directors** to secure API access, commercial terms, and integration roadmaps.
- Built landing page and conducted user research to refine feature priorities.

Automated Quarterly Reporting Agent (n8n + AWS) ***(Ongoing)***

Problem: Clients running survey campaigns needed monthly/quarterly trend analysis, but manual compilation took **60–80 hours per month** (equivalent to 1–2 FTEs).

Architecture:

- **50-step n8n workflow** orchestrating data ingestion, transformation, and reporting.
- **Custom JavaScript Nodes** (proprietary logic) for statistical analysis and trend detection.
- **Data Pipeline:** Survey platform → MongoDB → aggregation → AWS Lambda for PDF generation.
- **Storage:** PDFs saved to S3; AWS CloudFront CDN for fast downloads.
- **Distribution:** React UI for self-service downloads; automated email delivery (SES).

Key Components:

1. **Data Collection:** Fixed-period ingestion (monthly, quarterly); de-duplication and validation.
2. **Trend Analysis:** Compares current period against historical baselines; flags improving/declining metrics.
3. **AI-Powered Insights:** Uses OpenAI API to generate actionable suggestions (e.g., "Response rate down 12%; consider shortening survey").
4. **PDF Generation:** Dynamic report with charts, tables, and recommendations; branded with client logo.

Metrics & Outcomes:

- **Saves 60–80 hours per month** (2 FTEs freed for higher-value work).
- **Reduced report delivery time** from 2–3 days to minutes.
- **100% accuracy** (no manual compilation errors).

TabiSurvey – Advanced Survey Platform MVP ***(Ongoing)***

Problem: Clients needed a modern, flexible survey builder with rich question types, RBAC, and advanced analytics—faster than building from scratch.

Architecture:

- **Frontend:** Vue 3 (Composition API) + Vite (fast dev experience) + Pinia (state management).

- **UI Components:** Tailwind CSS for rapid styling; Material UI for pre-built accessibility.
- **Backend:** Supabase (PostgreSQL + realtime subscriptions) for rapid API iteration.
- **Database Schema:** Surveys, questions (multiple types), responses, user roles, access control.

Features:

- **Survey Builder:** Drag-and-drop interface; support for text, multiple-choice, emoji reactions, star ratings.
- **RBAC:** Role-based access (admin, editor, viewer); team collaboration.
- **Analytics Dashboard:** Real-time response counts, sentiment analysis, comparative trends.
- **Data Export:** CSV/JSON export for integration with BI tools.

Metrics & Outcomes:

- **MVP shipped in 3 weeks** (typical build time: 8–12 weeks = 4x faster).
- Production-ready performance (Lighthouse 95+).
- Ready for immediate client onboarding and feedback iteration.

TECHNICAL LEAD & PLATFORM ARCHITECT

EnduroSat | Sofia, Bulgaria | *December 2022 – December 2025*

(Official title: Product Manager; presented as Technical Lead & Platform Architect due to hands-on engineering contribution.)

Led the **complete architectural overhaul** of EnduroSat's core Satellite Operations Platform, stabilizing a fragile legacy system into a **high-performance, scalable, maintainable platform** serving mission-critical operations.

Satellite Operations Platform V2 – Complete Rewrite

Problem: The legacy platform was unreliable, difficult to maintain, and lacked features needed for modern satellite operations. Rewrite required from scratch.

Architecture (Before → After):

Aspect	Before	After
Frontend Framework	Legacy jQuery/Backbone	**React** (Hooks, Context)
State Management	Scattered global state	**Redux** (normalized schema, dev tools)

Aspect	Before	After
Styling	Inline CSS, BEM	**Tailwind CSS** + Material UI components
Build Tool	Webpack (slow dev)	Vite (instant HMR)
Performance	Slow telemetry rendering	**WebSockets** for real-time updates; memoization
AWS Deployment	Manual deploys	GitHub Actions CI/CD (dev/staging/prod)

Key Features Implemented:

1. **Live Ground Track:** Real-time visualization of satellite orbital path; WebSocket updates from server; interactive 3D globe (Cesium.js or similar).
2. **Command Center:** Unified interface for scheduling satellite commands; form validation; error handling; command history.
3. **Pass Schedule:** Display upcoming satellite passes; time-to-LOS (loss of signal) countdown; predictive analytics.
4. **Telemetry Dashboard:** Real-time sensor readings (temperature, power, communication status); historical trends; alerting.

Metrics & Outcomes:

- **Performance improvement:** Reduced initial load time from 8s → 2s; live telemetry refresh from 5s delay → <500ms.
- **Reliability:** Platform stabilized from frequent crashes to 99.9%+ uptime during mission-critical operations.
- **Development velocity:** New feature delivery improved 40% due to cleaner architecture.

Component Library & Design System Centralization

Problem: EnduroSat had multiple internal "UI Ops" products (monitoring dashboards, mission planning tools, etc.), each with its own button styles, colors, and layouts. Code duplication was rampant; bugs were fixed in one product but not others.

Solution: Built a **centralized Storybook component library** and enforced brand guidelines across all products.

Deliverables:

- **Storybook:** Interactive component documentation; 30+ reusable components (Button, Input, Modal, Table, Chart wrapper, etc.).

- **Brand Guidelines:** Centralized color palette, typography, spacing rules, and accessibility standards (WCAG 2.1 AA).
- **Testing:** Unit tests for all components (Jest); visual regression tests.
- **CI/CD Integration:** Design system changes trigger automated tests before merge.

Adoption & Impact:

- **20–25% codebase reduction** across all 5 internal products (duplicate component logic centralized).
- **Bug reduction:** Shared components tested once; fixes applied globally.
- **Onboarding:** New developers can spin up UI in hours, not days.
- **Maintenance burden:** 1 design system team manages components for all products.

Customer Success Portal (MyEnduroSat)

Problem: Customers had no visibility into their satellite health, service updates, or support status. Support tickets were high due to lack of self-service options.

Solution: Built a customer-facing portal using the new design system.

Features:

- **Dashboard:** Satellite health overview, last contact time, upcoming passes, mission status.
- **Service Updates:** Real-time notifications of platform maintenance, new features, and bugs.
- **Support Portal:** Self-service ticket submission, status tracking, knowledge base search.
- **Billing & Usage:** Usage metrics, invoice history (if applicable).

Metrics & Outcomes:

- **Support tickets reduced by ~20%** (self-service answered most common questions).
- **Customer satisfaction:** NPS improved due to transparency and self-service empowerment.
- **Operational efficiency:** Support team freed up for higher-value, complex issues.

AWS Infrastructure & CI/CD

Services Used:

- **S3:** Frontend static assets (HTML/CSS/JS bundles).
- **CloudFront:** CDN distribution; global caching; DDoS protection.
- **EC2:** Backend API servers (Node.js); auto-scaling groups.
- **Cognito:** User authentication; session management; multi-factor authentication.

- **RDS:** PostgreSQL database; automated backups; read replicas.
- **GitHub Actions:** Build → Test → Deploy pipeline; environment-specific configurations.

SENIOR FRONTEND ENGINEER

DevCloud BG | Plovdiv, Bulgaria | *2019 – 2022*

Industrial robotics company manufacturing silicon-wafer processing systems. Built the **Robot Control Center (RCC)**, a web application enabling operators to control expensive, complex machinery through an intuitive no-code interface.

Robot Control Center (RCC) – WebSocket-Based Control System

Problem: Operating industrial robots required deep technical knowledge; operators needed weeks of training. Manual command input was error-prone and slow.

Solution: Built a visual "block puzzle" workflow editor that abstracts robot commands into draggable blocks.

Architecture:

- **Frontend:** React + TypeScript; WebSockets for persistent, low-latency communication.
- **Workflow Builder:** Drag-and-drop interface; block types (Move, Wait, Sense, Conditional, Loop).
- **Execution Engine:** Translates block sequences into robot bytecode; sends via WebSocket.
- **Real-Time Feedback:** Robot status updates streamed back (position, temperature, errors).

Key Technical Challenges Solved:

1. **Low-Latency Control:** Commands must execute within 100ms of user interaction (safety-critical). Implemented WebSocket compression and binary protocol.
2. **Offline Support:** Robot can cache queued commands during network loss; syncs on reconnect.
3. **Safety Validation:** Blocks are validated before execution (no invalid sequences, no out-of-bounds moves).

Metrics & Outcomes:

- **Operator training time reduced** from 4 weeks → 2 weeks.
- **Error rate reduced** by 90% (no manual command syntax errors).
- **Throughput increased** by ~40% (faster command execution).
- **On-site deployment:** Traveled to China for 1 month; deployed, tested, and trained operators in live manufacturing environment.

SOFTWARE ENGINEER

Aucoda | Manchester, UK | *2017 – 2019*

Contributed to a **cross-platform application development platform** that compiles high-level source code into production-ready iOS, Android, and web applications.

AU Programming Language – OAuth & Cross-Platform Integration

Problem: Generated apps needed secure authentication; implementing OAuth separately for each platform introduced inconsistencies and bugs.

Solution: Built a unified OAuth handler in the AU language runtime that generates identical auth logic across all platforms.

Work:

- **OAuth 2.0 Implementation:** Authorization code flow, token refresh, secure storage of credentials.
- **Platform Bridging:** JavaScript (for web) ↔ native Objective-C (for iOS) interop; custom serialization.
- **Syntax Design:** Proposed language syntax for OAuth configuration (host, client ID, scopes, redirect URI).
- **Testing:** Unit tests across platforms; integration tests with real OAuth providers (Google, GitHub).

Outcome:

- Generated apps shipped with OAuth support; reduced auth-related bugs by ~80%.

EDUCATION

Computer Science
University of Manchester | *2015 – 2017*

LANGUAGES

- **Bulgarian** – Native fluency
- **English** – Professional fluency (written, verbal, technical)

KEY ACHIEVEMENTS & QUANTIFIED IMPACT

Achievement	Metric	Impact
Prezaredi.bg Launch	20 clients, 400 fleet cards, 99%+ uptime	Enabled digital fuel payments at scale
Automated Reporting Agent	60–80 hours/month saved	2 FTEs freed for strategic work
TabiSurvey MVP	3 weeks to production	4x faster than industry standard
EnduroSat Platform V2	8s → 2s load time; 5s → <500ms telemetry	75% faster; mission-critical stability
Design System	20–25% code reduction	Fewer bugs, faster development
MyEnduroSat Portal	20% reduction in support tickets	Improved customer self-service
RCC Deployment	4 weeks → 2 weeks training; 90% error reduction	Operator productivity up 40%